

实验一：无线传感器网络开发环境

一、实验目的

1. 了解 WINAVR 集成开发环境
2. 学习使用 AVR Studio 下载程序
3. 学习使用 AVR 单片机定时器/计数器 0 查询工作方式的基本设置和应用

二、实验环境

1. PC 机 一台
2. GAINS-3/GAINZ 教学开发平台 一套
3. WINAVR V4.07
4. AVRStudio

三、实验内容和要求

1. 安装 Win AVR, AVR Studio
2. 编写一个走马灯程序并添加到工程中。使用定时器/计数器 0 定时控制三个灯轮流闪烁。采用查询的方式等待定时时间到达。
3. 设置 IDE 编译环境
4. 使用 makefile
5. 编译程序
6. 正确连接硬件
7. 设置编程程序的参数
8. 编程程序

四、实验指导

(1) 安装

运行WinAVR-20040720-install.exe,选择中文简体语言,按照默认选择,点击下一步,选择“我同意”,然后安装在您指定的位置,最后选择所需要安装的组件,建议选择全部安装。

(2) 使用

安装结束之后,选择程序->WINAVR->Programmers Notepad [WinAVR],即可打开WINAVR编译环境Programmers Notepad。现在便可以在该环境下进行编译,也可以选择其他的编译环境,如Ultraedit或者Source Insight,在这里就不赘述了。

(3) 编写一个简单的 C 程序

启动 Programmers Notepad 集成开发环境,选择 File->New->Project,新建一个工程,并保存在您的工程文件夹中。

因为在编写与硬件密切相关的单片机程序时通常需要想象编译器是如何将这些代码翻译成汇编程序的,因此我们推荐使用 C。选择 File->New->C/C++,并选择 Save as 保存为一个新的文件,假设命名为 test1。如图 1-1 所示,点击左侧窗口中新建的工程 AVR,选择 Add Files,

在文件对话框中，选择 test1.c,单击打开，即可将该文件添加到工程文件中。此时，便就可以在 test1.c 中编辑程序了。在这里，我们给出一个实例程序，这是一个使用定时器/计数器 0 定时控制三个灯轮流闪烁，并采用查询的方式等待定时时间到达的程序。

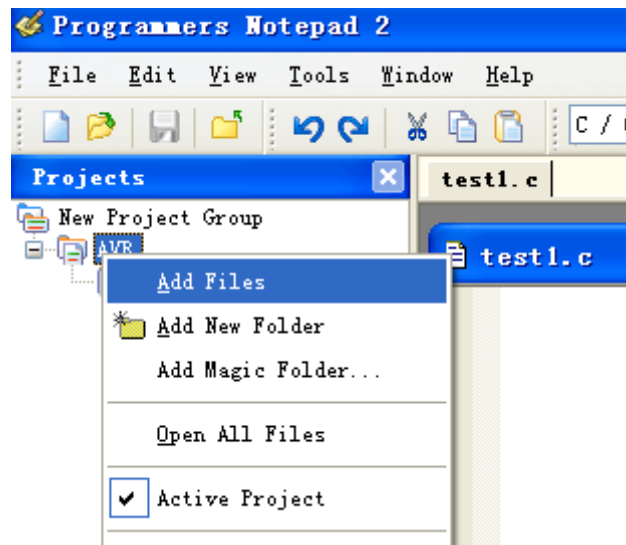


图 1-1 添加源程序文件

程序清单 1-1 走马灯代码

```
/*  
文件名： test3.c  
功能： 使用定时器 0 实现定时 0.5 秒的走马灯实验  
说明： 时钟： 7.3728MHz  
*/  
  
#include <avr/io.h>  
  
#define SET_YEL_LED PORTA&=~_BV(PA0) //PA0 口输出低电平，黄灯亮  
#define CLR_YEL_LED PORTA|=_BV(PA0) //PA0 口输出高电平，黄灯灭  
  
#define SET_GEN_LED PORTA&=~_BV(PA1) //PA1 口输出低电平，绿灯亮  
#define CLR_GEN_LED PORTA|=_BV(PA1) //PA1 口输出高电平，绿灯灭  
  
#define SET_RED_LED PORTA&=~_BV(PA2) //PA2 口输出低电平，红灯亮  
#define CLR_RED_LED PORTA|=_BV(PA2) //PA2 口输出高电平，红灯灭  
#define uchar unsigned char  
  
void Delay(void);  
void FlashLedYel(void);  
void FlashLedRed(void);  
void FlashLedGen(void);
```

```

/*****
*名称: main()
*功能: 走马灯程序, 让三个灯轮流闪烁
*入口参数: 无
*出口参数: 无
*****/

int main( void )
{
    PORTA=_BV(PA0)|_BV(PA1)|_BV(PA2);
    DDRA=_BV(PA0)|_BV(PA1)|_BV(PA2);    //设置 PA0,PA1,PA2 为输出口
    while(1)
    {
        FlashLedRed();
        FlashLedYel();
        FlashLedGen();
    };
}

/*****
*名称: Delay()
*功能: 初始化 I/O 及定时器 0, 定时时间为 0.5s
*入口参数: 无
*出口参数: 无
*****/

void Delay()
{
    uchar i=0;
    TCNT0=0; // T/C0 开始值
    TCCR0=_BV(CS02); // 预分频 ck/64 ,计数允许

    //查询定时器方式等待 0.5 秒
    //7372800 /64 /256 /225 = 2Hz
    for(i=0;i<225;i++)
    {
        loop_until_bit_is_set(TIFR,TOV0);
        //写入逻辑1 清零 TOV0 位
        TIFR|=_BV(TOV0);
    }
}

```

```

/*****
*名称:  FlashLedYel()
*功能:  控制黄灯闪烁
*入口参数:  无
*出口参数:  无
*****/

void FlashLedYel(void)
{
    SET_YEL_LED;
    Delay();
    CLR_YEL_LED;
    Delay();
}

/*****
*名称:  FlashLedRed()
*功能:  控制红灯闪烁
*入口参数:  无
*出口参数:  无
*****/

void FlashLedRed(void)
{
    SET_RED_LED;
    Delay();
    CLR_RED_LED;
    Delay();
}

/*****
*名称:  FlashLedGen()
*功能:  控制绿灯闪烁
*入口参数:  无; 出口参数:  无
*****/

void FlashLedGen(void)
{
    SET_GEN_LED;
    Delay();
    CLR_GEN_LED;
    Delay();
}

```

如图1-2所示，通过点击Tools->[WINAVR] Make All编译文件。

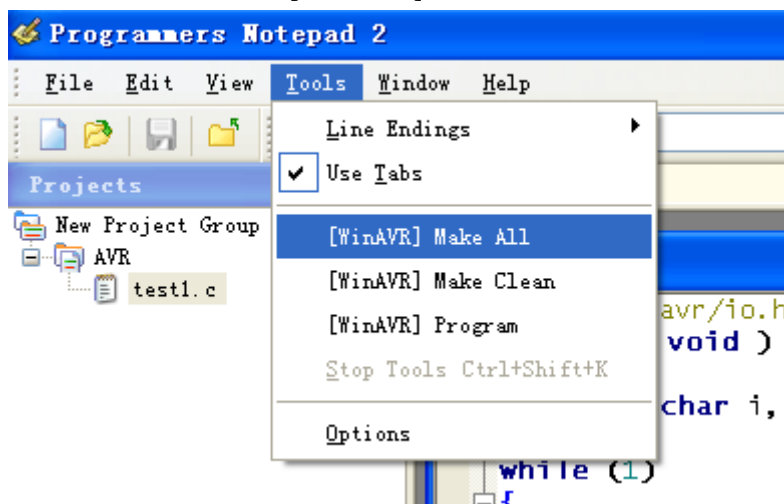


图 1-2 Make C 语言源程序文件

此时会出现如图 1-3 所示对话框

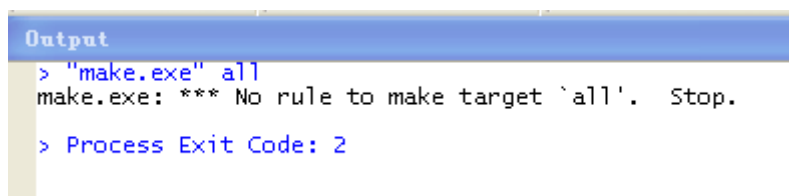


图 1-3 缺少 makefile 文件错误

“No rule to make target `all`. Stop.”的意思是我們没有设置项目管理器，下面让我们了解一下WINAVR的项目管理器Makefile。

(4) 用 Make 管理项目

通常一个编译器（泛指高级语言编译器、汇编器、连接器等等）、项目管理和文本编辑器构成一个完整的编程环境。

其中项目管理器负责为每一个源程序文件调用编译器生成目标文件后用连接器将它们组合在一起生成可执行文件。例如Keil uVision 进行编译时将项目中的每一个源程序文件进行编译后生成对应的.obj 文件，之后将这些目标文件连接到一起生成可执行（例如iHex 格式）文件。这是对编译过程的粗略的描述，Keil uVision 掩盖了很多的细节，使用过早期C51编译器的读者对此一定有所了解。

WINAVR没有像Keil uVision那样集成项目管理的IDE，所以我们需要写一个叫做makefile的文件来管理程序的编译连接。makefile是个脚本文件，一个标准的可执行文件make.exe负责解析它并根据脚本内容来调用编译器、连接器或其它的工具，最终生成可执行代码文件。每次调用make时它会比较目标文件与源文件的更新时间，如果源文件比目标文件还要新它会执行makefile内的相关指令更新目标文件，如果目标文件与源文件一样新它就跳开这个源文件的编译，避免重复工作，这对于一个较大的工程是个节省时间的有效方法。

WINAVR包中有一种简单的makefile文件生成工具叫mfile，利用它我们可方便的生成合适的makefile。选择程序->WINAVR->MFile，即可打开makefile，它的运行界面如图1-4所示。

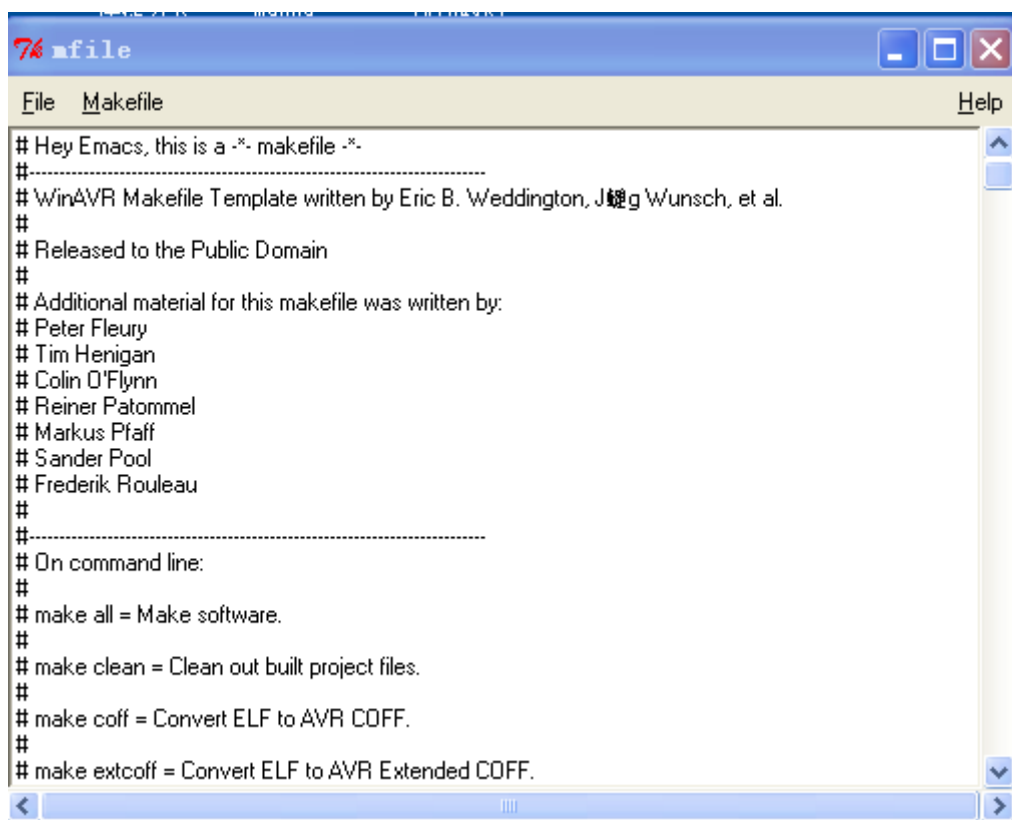


图 1-4 Mfile运行界面

MCU type根据所选择的处理器进行设置，这里我们选择ATmega128，设置如图1-5。

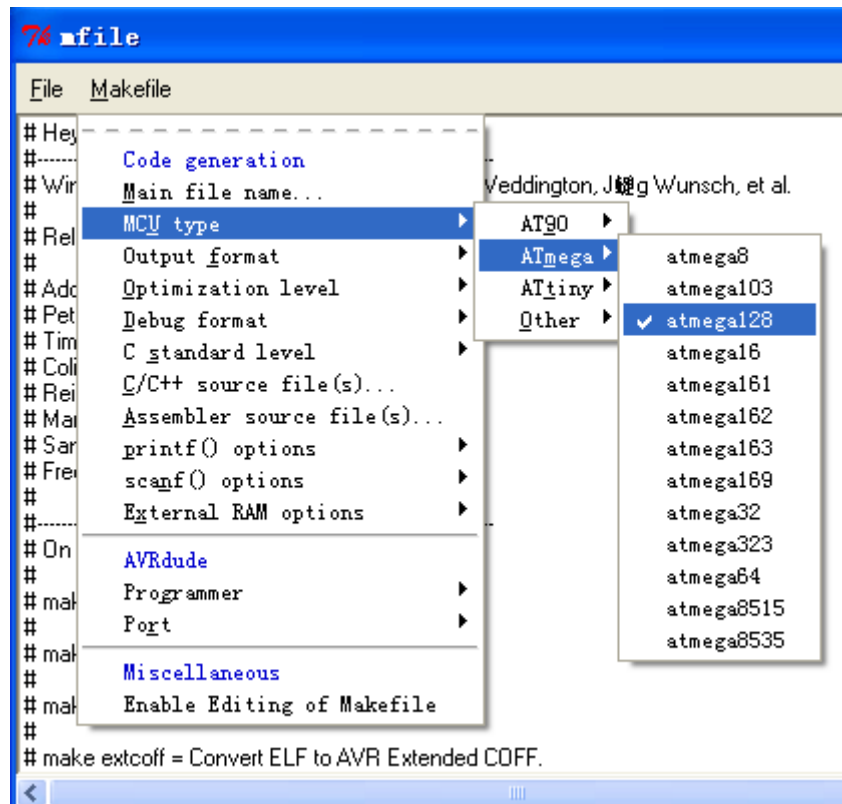


图 1-5 设定MCU type

Main file name...菜单指定主程序文件，它将决定主源程序文件名及输出文件名。选择

该选项，在对话框中输入您主程序存放的文件名，这里我们使用test1，注意，请不要加后缀，如.c等，如图1-6所示。



图 1-6 输入Main file name

Output format菜单项用于选择最终生成的可指行代码格式，根据编程器支持格式选择即可。如图1-7所示选择ihex。

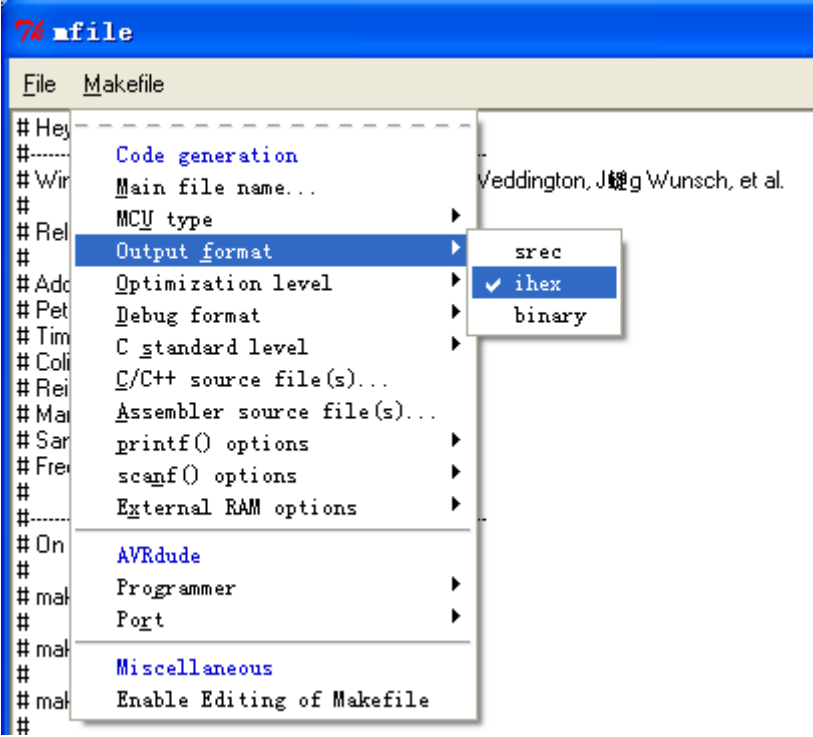


图 1-7 设定Output format

Optimization level指定C 代码的优化级，s 代表按最小代码量编译。设置如图1-8所示。

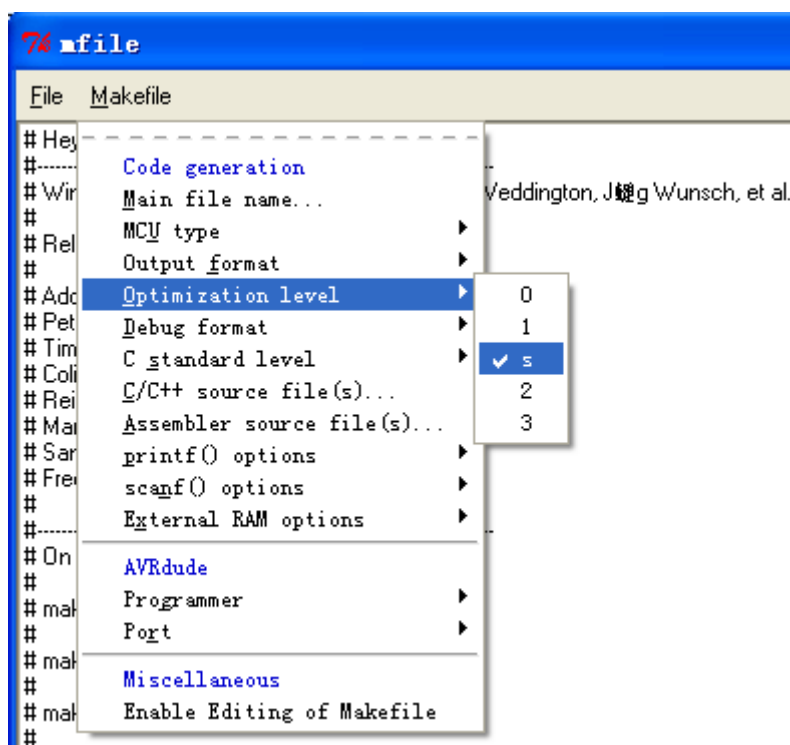


图 1-8 设定Optimization level

C/C++ source file(s) 和Assembler source files(s) 用于在项目添加其它C、C++、和汇编程序源文件。C/C++ source file(s)为例，其对话框如图1-9所示。由于我们目前的工程中只有一个文件，因此可以不设置。

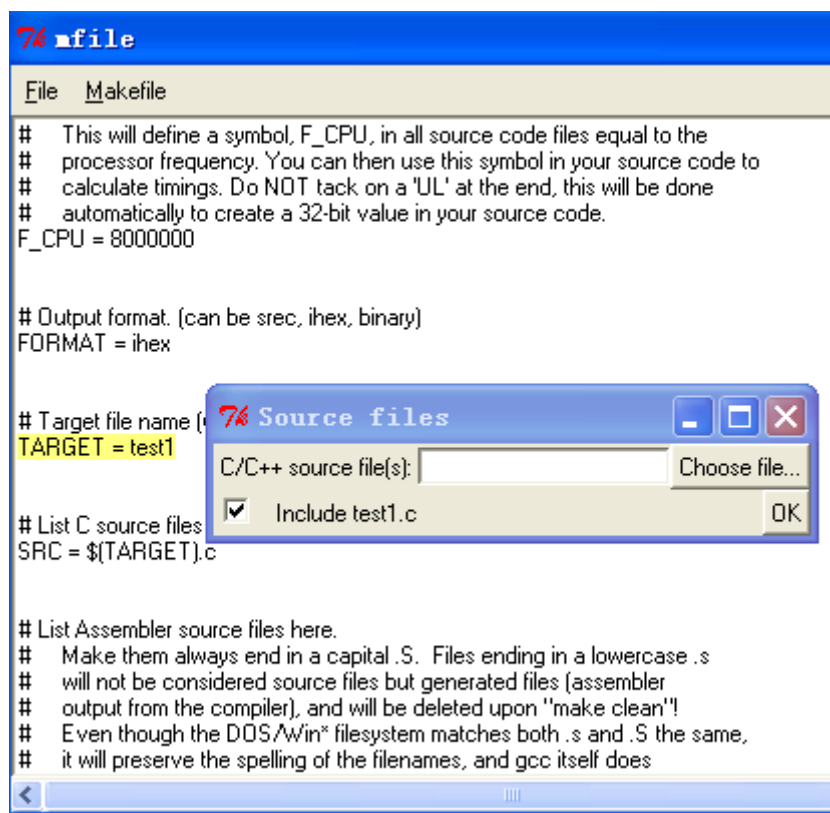


图 1-9 添加C/C++ source file(s)

通常选择了以上几项就可以了，在编写程序的过程中往往需要在项目中添加源文件，这时只需在生成的makefile中SRC变量后列出源文件名即可。

当一切设置好之后，选择File→Save As, 将设置好的makefile文件存放到工程文件夹中，此时，项目管理器就已经设置好了。此时，再点击Tools→[WINAVR] Make All编译文件，就可以在Output中看到正确编译的结果。编译之后文件就可以下载到硬件中了。

(5) 安装 AVR Studio

点击 AVRStudio4.07 文件夹中的安装文件，开始安装程序。

(6) 硬件平台的搭建

无线传感器网络系列节点采用 JTAG 口来编程。JTAG 口的管脚顺序与标准 JTAG 口的顺序略有不同，所以编程的时候需要一个转接板，此转接板随仿真器一起提供。转接板如图 1-10。编程时数据线连接顺序如下：有 JTAG 字样的一端通过数据线与节点相连，有 GAIN 字样的一端通过数据线与仿真器相连。

注意：由于仿真器有自保护功能，所以节点上的 JTAG 口没有做防插错处理，如果方向插错，仿真器无法识别，这时候只需要调换一下插头方向就可以了。

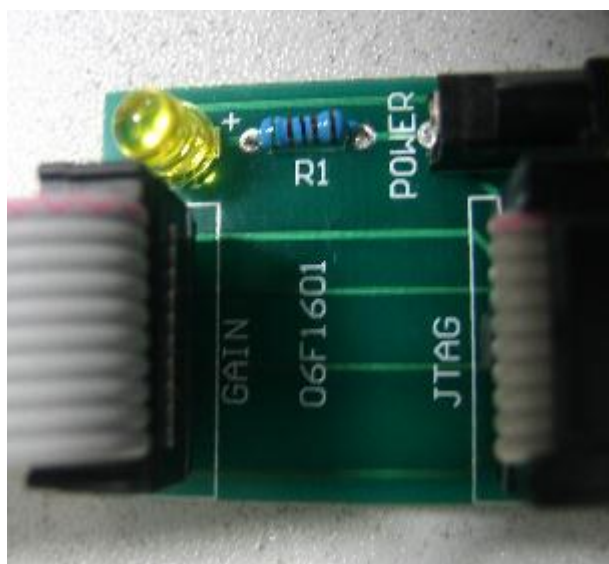


图 1-10 转接板

仿真器与 PC 的接口有两种：

1) 串口：通过串线与 PC 相连接的时候，仿真器需要外接电源，电源接口见图 1-10 的 POWER 部分，电源已经随仿真器一起提供，如果电源正常，电源指示灯会亮。在有外接电源的状态下，仿真器给节点编程的时候不需要打开节点不需要打开。

2) USB 口：使用 USB 线与 PC 相连时，PC 上要安装一个驱动，驱动程序放在光盘的“\Software\驱动程序\FT232 Driver”文件夹。转接板上要接外接电源，在有外接电源的状态下，仿真器给节点编程的时候节点不需要上电。AVRstudio 只认 1-10 号串口，如果虚拟出的串口号大于 10，可以在设备管理器里把串口号修改成小于 10 就可以了。串口和 USB 口不能一起使用。

到这里一些准备工作基本完成，硬件平台已经搭建完成，下面就可以通过 AVR Studio 软件向节点下载程序了。软件启动后会显示一个欢迎界面，并且会显示最近使用过的文件或工程以供用户选择，选择 Cancel，接着进入了该软件的主界面，在该界面下开始下载程序。

(7) 设置参数

选择程序->Atmel AVR Tools->AVR Studio 4，打开 AVR Studio。

首先要保证硬件的连接正确无误（现阶段传感器子板可以不插，传感器子板主要在采集数据时才会使用到），打开节点的电源开关，否则会检测不到节点而导致编程失败。下面将编程的步骤详细列举出来，供大家参考。

Step 1: 进入编程程序的界面

如图 1-11 所示，选择 Tools //STK500/AVRISP/JTAG ICE //STK500/AVRISP/JTAG ICE...

或者直接点击界面上的  按钮，会弹出如图 1-12 所示一个工作界面，这个界面是下载程序的界面，下面的一些工作都在该界面下展开。

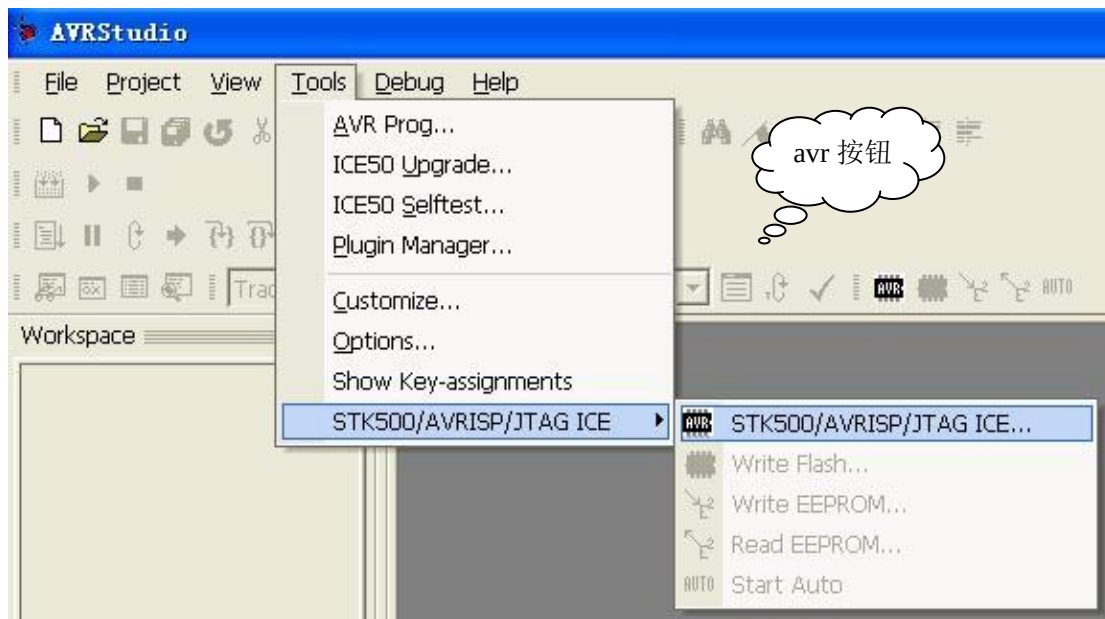


图 1-11 进入编程程序的界面

Step 2: 设备和一些参数的设置

此时已经进入了编程程序的界面，可以看到有 6 个选项，分别是 Program, Fuses, LockBit, Advanced, Board 和 Auto，需要用到的是 Program 与 Fuses，后面的几项保持默认。在 Program 选项下首先需要在 Device 的下拉列表下面选择单片机的型号，节点使用的是 ATmega128，需要选中对应的设备。Programing mode 选项中有两项是可选的，它们是 Erase Device Before 和 Verify Device After Program，如果选中了第一个，在编程程序时会先将 ATmega128 的 128k 的 flash 全部擦除，然后再向里面下载程序。第 2 个选项表示在编程程序完成以后进行一些检校和验证的工作，确保编程的数据是无误的。建议这两个选项都选上，尽管这样会使编程的时间更长，但是这对于确保数据的正确性方面却是很必要的。在 Advanced 选项卡中选择串口速率，如图 1-13 所示，建议不要选择过高的速率，速度过高编程的时候容易出错。

注意：GAINS-3 系列节点在出厂之前已经将熔丝位设置好，Fuses 标签页中，用户无需修改。对于需要修改熔丝位的用户，建议在仔细阅读相关资料后执行，否则易发生芯片被锁定等事故。

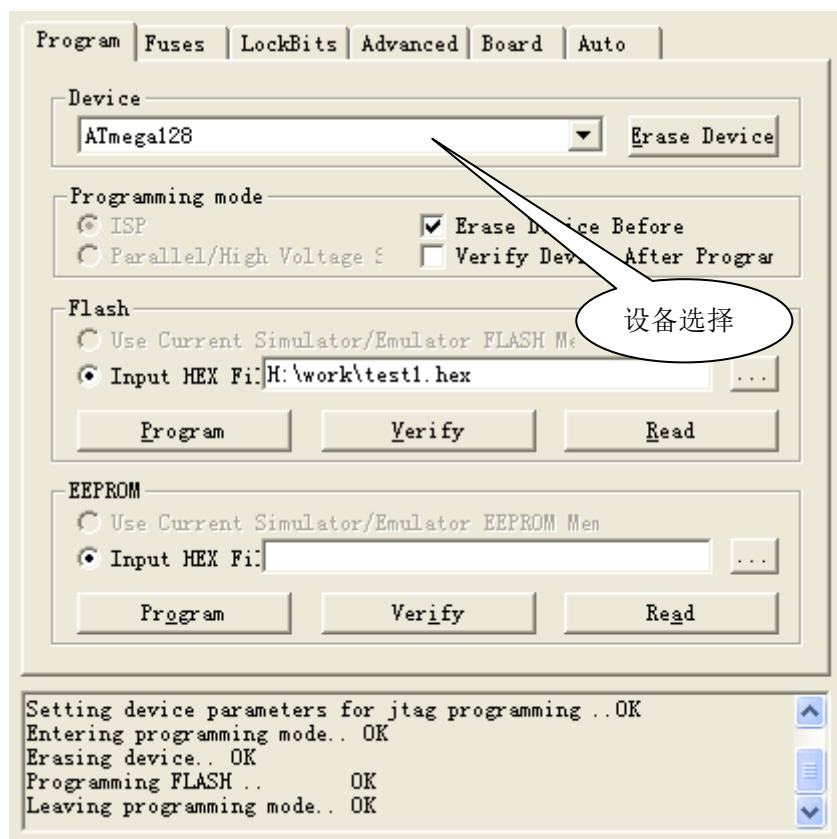


图 1-12 下载界面

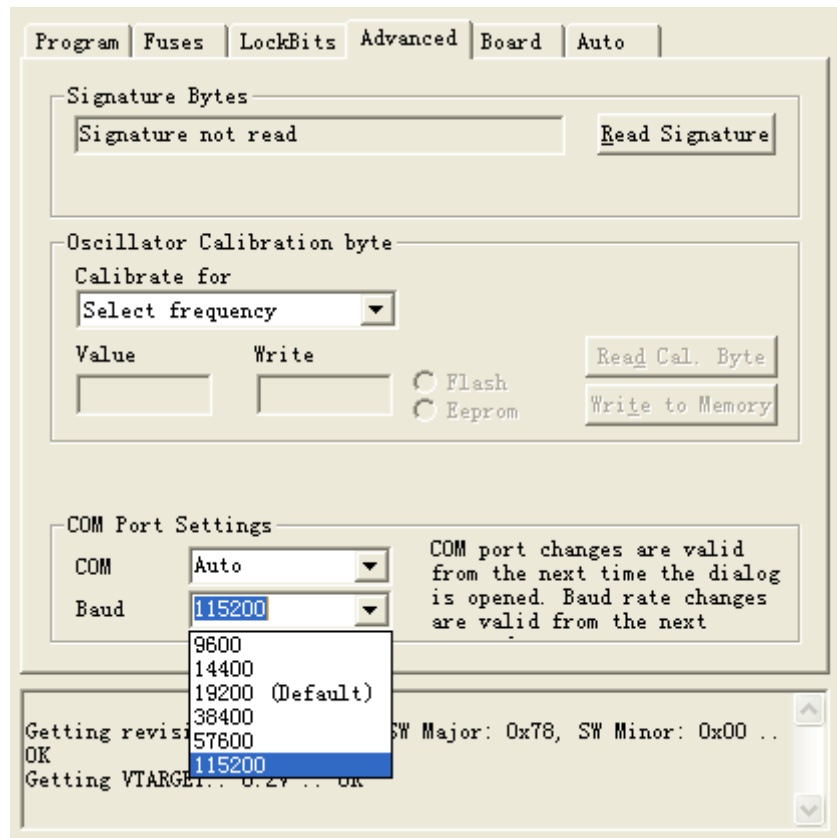


图 1-13 在 Advanced 选项卡中选择串口速率

(8) 下载程序

如图 1-14 图 所示, 选中了相关的 test1.hex 文件后, 点击 Flash 框中的 Program 按钮就可以将 hex 文件下载到 ATmega128 中了。程序下载成功之后可以看到实验开发板上的灯。

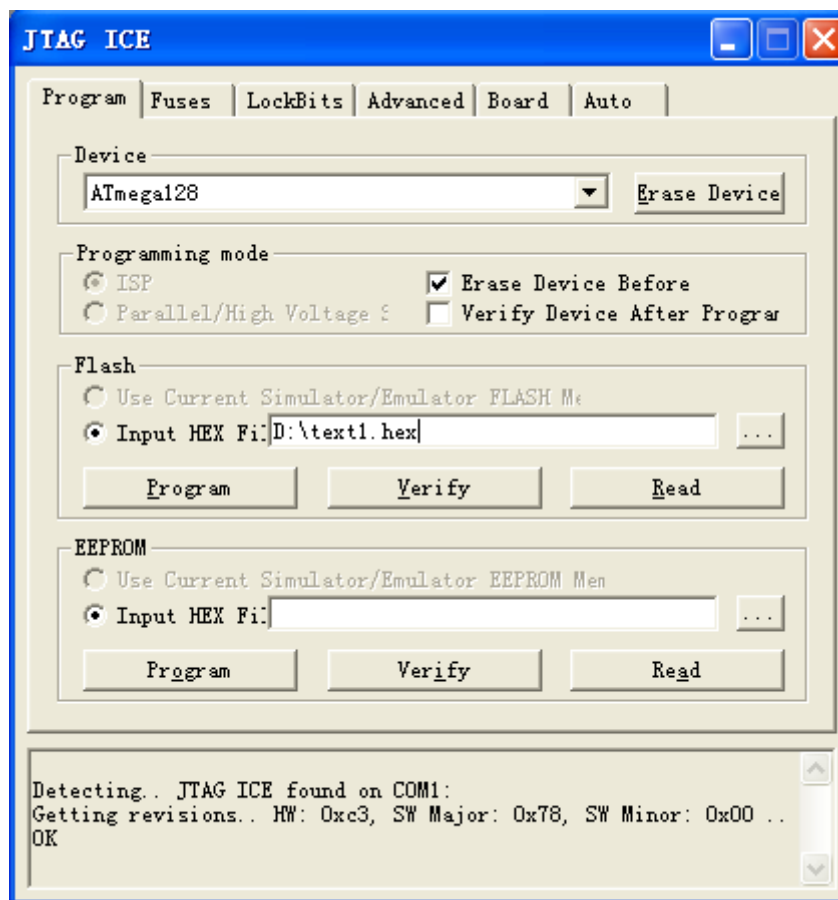


图 1-14 下载程序

实验二：点到点无线通信实验

一、实验目的

1. 进一步掌握 WINAVR 集成开发环境
2. 进一步掌握 AVR Studio 下载程序的使用
3. 培养开发 CC1000 射频驱动的能力

二、实验设备

1. PC 机 一台
2. GAINS-3 教学开发平台 一套
3. 软件： WINAVR V4.07
4. AVRStudio

三、实验内容

在该实验中将需要两个节点，一个作为发送节点(node 节点)，一个作为接收节点(base 节点)。开始的时候两个节点的 CC1000 的状态设置为接收状态，发送节点通过时钟中断，定期的将 CC1000 的状态设置为发送状态，同时发送一个数据包，发送完数据包后转换为接收状态。CC1000 相关的配置方法在 CC1000 器件手册上面有详细的介绍，在我们的工程文件的 cc1000.c 中有相关的配置函数。实验中每隔 230ms 发送节点发送一个数据包，接收节点收到后将该数据包通过串口传给 PC 主机。串口的设置为波特率 57600，8 位数据位，1 位停止位，无校验位。

四、实验步骤

1. 打开文件夹“点到点通信实验”下面的工程文件，编译“base”和“node”文件夹下面的工程得到 hex 文件；
 - a) 文件夹中文件说明：cc1000.c 为 CC1000 驱动；fun.c 为管脚设置文件；uart.c 为串口驱动；led.c 为 LED 灯驱动。
2. 将 hex 文件下载到节点；
3. 将 base 节点通过串口线和 PC 主机，PC 主机运行串口调试助手，打开 base 节点和 node 节点，这时可以看到 node 节点发送过来的数据。数据包示例：8E 04 FF EE DD 7E。
 - a) 注意：使用 COM 口接收 node 节点发送过来的数据时，是直接将节点的 COM 口与计算机的 COM 口相连；
 - b) 请正确设置串口调试助手的参数。
4. 请修改程序，使得当 node 节点发送完一个数据包后，绿灯闪一下；而当 base 节点收到一个数据包后，红灯闪一下。
 - a) 修改好程序之后，必须重新 Make 目标文件；
 - b) 文件夹“点到点通信实验”下面已经有现成的 Makefile 文件，因此重新 make 以后就可以直接给节点编程。

五、实验参考程序

程序清单 2-1 main.c

```
/******  
文件名：main.c  
功能：普通节点主控模块（node）  
说明：  
*****/  
uint8_t buff[4]={0x04,0xff,0xee,0xdd}; //发送的数据包  
int main(void){  
    OSH_SET_PIN_DIRECTIONS();//引脚配置  
    uart_init(); //串口初始化  
    TimerStdControlInit(); //设置定时器  
    LedInit(); //指示灯初始化  
    LedRedOff();  
    LedGreenOff();  
    LedYellowOff();  
    PhysicalInit(); //物理层初始化  
    __asm volatile ("sei");//开全局中断  
    while(1);  
    return 1;  
}  
/******  
*功能描述：时间中断回调函数，启动无线发送功能  
*参数说明：无  
*返回值：  
*****/  
void Timer0_Fired(void){  
    PhysicalTxPkt(buff,4);//调用无线发送功能  
}  
/******  
*功能描述：单片机引脚配置  
*参数说明：无  
*返回值：  
*****/  
void OSH_SET_PIN_DIRECTIONS(void )  
{  
    OSH_MAKE_RED_LED_OUTPUT();  
    OSH_MAKE_YELLOW_LED_OUTPUT();  
    OSH_MAKE_GREEN_LED_OUTPUT();  
    OSH_MAKE_CC_PALE_OUTPUT();
```



```

} else if (state == IDLE) {                                     // 空闲
    bManchesterBad = *(volatile unsigned char *) (0x1A + 0x20) & 0x40; // CHP_OUT 位
    if ((!bManchesterBad) && (data == 0xaa || data == 0x55)) {
        PreambleCount++;
        if (PreambleCount > VALID_PRECURSOR) {                // 接收到足够的导频字节
            state = SYNC_START;
        }
    } else {
        PreambleCount = 0;                                     // 导频计数置 0
    }

} else if (state == SYNC_START) {                               // 接收同步字
    uint8_t i;
    if (data == 0xaa || data == 0x55) {
        SOFCount = 0;                                         // 再收到同步字节则丢弃
    } else {                                                    // 进行同步
        uint8_t usTmp;
        SOFCount++;
        switch (SOFCount) {
            case 1:
                RxShiftBuf.MSB = data;
                break;
            case 2:
                RxShiftBuf.LSB = data;
                if (RxShiftBuf.W == SYNC_WORD) {
                    state = RECEIVE;
                    RxBitOffset = 0;                            // 通知上层检测到起始符号且同步完成
                    if (PhysicalStartSymDetected() == FAIL) {
                        RadiocontrolIdle();
                    }
                }
                break;
            case 3:
                usTmp = data;
                for(i = 0; i < 8; i++) {
                    RxShiftBuf.W <<= 1;
                    if(usTmp & 0x80)
                        RxShiftBuf.W |= 0x1;
                    usTmp <<= 1;
                    if (RxShiftBuf.W == SYNC_WORD) {

```

```

        state = RECEIVE;
        RxBitOffset = 7-i;
        RxShiftBuf.LSB = data; // 通知上层检测到起始符号且同步完成
        if (PhysicalStartSymDetected() == FAIL) {
            RadiocontrolIdle();
        }
        break;
    }
    break;
default:
    RadiocontrolIdle();
    break;
}
}

}else if (state == RECEIVE) { // 根据偏移量接收数据
    char Byte;
    RxShiftBuf.W <=<=8; // 移去已经接收的字节
    RxShiftBuf.LSB = data; // 存入刚刚收到的字节到低八位
    Byte = (RxShiftBuf.W >> RxBitOffset); // 根据偏移量取出字节

    PhysicalRxByteDone(Byte); // 通知上层处理接收到的字节
}
}

/*****
*功能描述：射频控制进入空闲状态，CC1000、SPI 进入接收状态
*参数说明：无
*返回值： 成功
*****/

result_t RadiocontrolIdle()
{
    if (state == IDLE)
        return SUCCESS;
    if (state == SYNC_START) {
        state = IDLE;
        return SUCCESS;
    }
    if (state == SLEEP) { // 打开 CC1000
        CC1000ControlStdControlStart();
    } else {

```

```

        * (volatile unsigned char *) (0x0D + 0x20) = 0x00;    // 关 SPI 和 SPI 中断
    }
    CC1000ControlRxMode();                                     // CC1000 设为接收模式
                                                           // 配置 SPI 为输入

    OSH_MAKE_MISO_INPUT();
    OSH_MAKE_MOSI_INPUT();
    * (volatile unsigned char *) (0x0D + 0x20) = 0xc0;        // 开 SPI 和 SPI 中断

    state = IDLE;
    return SUCCESS;
}

/*****
*功能描述：启动发送数据包的流程，并准备要发送的导频字节，CC1000、SPI 进入发送态
*参数说明：无
*返回值：成功
*****/
result_t RadiocontrolStartTx()
{
    char temp;
    * (volatile unsigned char *) (0x0D + 0x20) = 0x00;    // 关 SPI 中断
    if (state == SLEEP) {                                   // 如果睡眠则唤醒
        CC1000ControlStdControlStart();
    }
    nextByte = 0xaa;                                       // 缓存下一个要发送的字节
    txCount = 2;
    temp = * (volatile unsigned char *) (0x0E + 0x20);    // 清空 SPSR 寄存器
    * (volatile unsigned char *) (0x0F + 0x20) = 0xaa;    // 将要发送的第一个字节写入 SPDR

    //进入发送模式
    CC1000ControlTxMode();
    OSH_MAKE_MISO_OUTPUT();
    OSH_MAKE_MOSI_OUTPUT();
    * (volatile unsigned char *) (0x0D + 0x20) = 0xc0;    // 开 SPI 中断

    state = TRANSMIT;
    return SUCCESS;
}

```

程序清单 2-3 physical.c

```
/**
 * 文件名: physical.c
 * 功能: 物理层控制模块
 * 说明:
 */

/*
 * 功能描述: 物理层发送数据包接口函数, 进入发送状态, 启动整个发送流程
 * 参数说明: packet: 数据包地址 length: 物理层要发送的字节数
 * 返回值: 无
 */
result_t PhysicalTxPkt(void* packet, uint8_t length)
{
    state = TRANSMITTING;
    RadiocontrolStartTx();
    sendPtr = (char*)packet;
    pktLength = length;
    txCount = 0;

    return SUCCESS;
}

/*
 * 功能描述: 将收到的字节打包成数据包, 并发出包处理任务
 * 参数说明: 收到的字节
 * 返回值: 执行结果
 */
result_t PhysicalRxByteDone(char data)
{
    if (state != RECEIVING)
        return FAIL;

    if (recvCount == 0) {
        pktLength = (uint8_t)data; // 第一个字节为包长度
        // 得到包长度
        if (pktLength != 0x04) { // 对不同的数据包长度要进行修改
            RadiocontrolIdle(); // 转入空闲状态
        }
        state = IDLE;
    }

    recvPtr[recvCount] = data;
    recvCount++;
}
```

```

        if (recvCount == pktLength) {                                // 接收完成
            RadiocontrolIdle();                                       // 转入空闲状态
            state = IDLE;
            packet_received();                                         // 接收缓存可用
        }
        return SUCCESS;
    }

/*****
*功能描述:
*发送未完成时, 准备下一个要发送的字节, 发送完成时进入空闲状态并通知上层
*参数说明: 无
*返回值: 无
*****/

result_t PhysicalTxByteReady(void)
{
    // 射频控制模块获得数据包下一个字节
    if(state == TRANSMITTING) {
        RadiocontrolTxNextByte(sendPtr[txCount]);
        txCount++;

        if (txCount == pktLength) {                                // 发送最后字节
            state = TRANSMITTING_LAST;
        }

    } else if (state == TRANSMITTING_LAST) {
        state = TRANSMITTING_DONE;
    } else if (state == TRANSMITTING_DONE) {
        RadiocontrolIdle();                                         // 射频控制转入空闲
        state = IDLE;
    }
    return SUCCESS;
}

/*****
*功能描述: 检测到起始符号, 准备接收数据
*参数说明: 无
*返回值: 无
*****/

result_t PhysicalStartSymDetected(void)

```

```
{  
    // 物理层处于空闲状态时  
    if (state == IDLE ) {  
        state = RECEIVING;  
        rcvCount = 0;  
        return SUCCESS;  
    }  
    return FAIL;  
}
```

实验三： WSNs 指定路由多跳网络实验

一、实验目的

1. 掌握 GAINS433MAC 库的使用
2. 培养利用 GAINS433MAC 库开发高级通讯协议的能力
3. 学习通过 SNAMP 可视化后台软件观察网络的可视化结果。

二、实验设备

1. PC 机 一台
2. GAINS-3 教学开发平台 一套
3. 软件： WINAVR V4.07
4. AVRStudio

三、实验内容

使用 GAINS433MAC 库实现一个指定路由的多跳无线网络，将传感数据从远方的传感器节点发送到汇聚节点。汇聚节点通过串口将收集到的传感数据发送到 PC，通过 PC 上的 SNAMP 可视化后台软件观看光强的实时曲线以及路由情况。

四、实验步骤

1. 启动 Programmers Notepad，打开文件“指定路由实验”目录下的 router.pnproj 工程。
2. 在 app.c 中的 Init 函数中修改 OS_LOCAL_ADDRESS 的值，给其赋值 1(即 Sink 节点)，2, 3, 分别编译后获得 3 个.hex 文件，读者可以通过修改 POWERLEVEL 的值来修改天线的功率，从而控制传输距离，具体的参数设置见附录中的表 a-1。
3. 将.hex 文件下载到节点中。
4. 通过串口线连接 1 号节点到 pc，打开所有节点。
5. 启动 SNAMP 可视化后台软件，通过 SNAMP 可以观察到 3 号节点每次都通过 2 号节点转发数据包到 sink 节点。
6. 修改源代码，实现如下功能：
 - a) 当 node 节点发送完一个数据包后，绿灯闪一下；
 - b) 当 node 节点转发一个数据包后，黄灯闪一下；
 - c) 而当节点收到一个数据包后，红灯闪一下；
 - d) 对于一个由 4 个节点（序号分别为 1, 2, ..., 4）组成的网络，设置 ID=1 的节点为 Sink 节点，任意节点 m ($1 < m < 5$) 至 sink 节点的路由为 $m \rightarrow m-1 \rightarrow \dots \rightarrow 2 \rightarrow 1$ 。
 - e) 仅 ID 最大的节点才能发送感知到的数据。

五、GAINS433MAC 库介绍

GAINS-3 试验开发板最主要的功能是实现半双工的无线通信，为了方便读者的开发学习，提供了 MAC 层协议库，相关的函数以功能库的形式提供。提供的 MAC 层协议的库能够提供的功能包括 ACK 机制，射频睡眠机制以及透明的传输数据包的能力，采用了

CSMA/CA 技术，使用二进制退避算法实现冲突避免，利用 CRC-16 算法进行检校。

另外，还附带的提供了一些辅助的库和模块，这些库和模块包括控制 LED 的库（开源），控制传感器的库（开源），控制 ADC 的库（开源），控制 0 号计数器的库（开源），射频驱动的库，任务调度库（开源，相当于操作系统），功能函数模块（开源）以及串口驱动模块（开源），使用这些库和模块可以满足大部分的开发需要，而且还提供一个开发模板，使用该模板可以让读者的开发更方便，在模板的基础上读者可以添加自己需要的功能。图 3-1 是提供的模版的一个系统架构。表 3-1 表 3-1 是一些代码文件的清单。

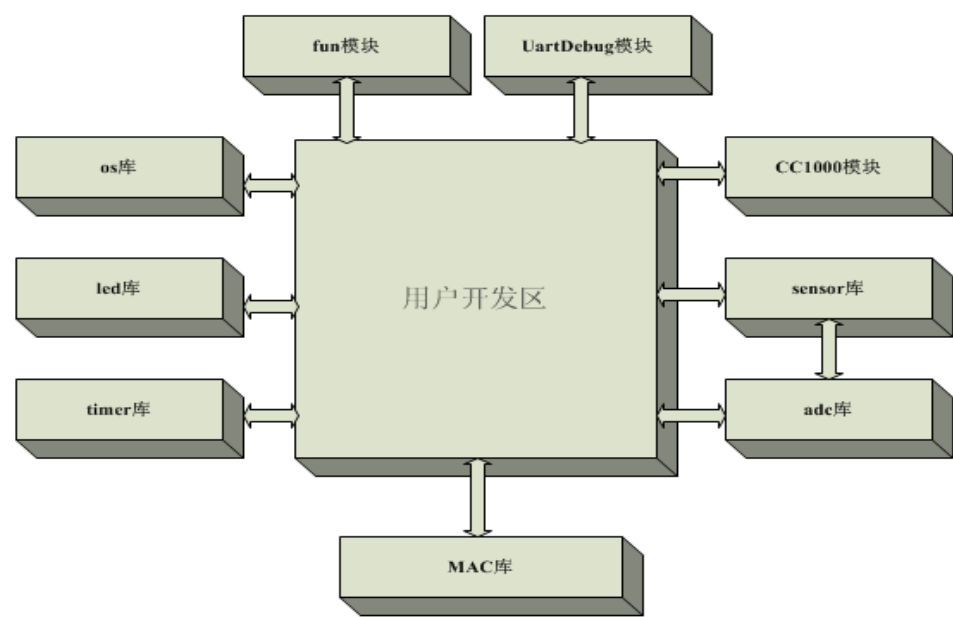


图 3-1 模板架构

表 3-1 代码文件清单

硬件驱动	其余驱动	led.h	实现对 LED 的控制	辅助模块
		led.c		
		adc.h	实现对 ATmega128 单片机 ADC 转换器的控制	
		adc.c		
		sensor.h	实现对外部传感器的控制，包 括对其电源管理和数据的提 取。	
		sensor.c		
		os.h	提供基本的任务调度，可以看 作是一个小型的操作系统	
		os.c		
		fun.h	实现对 ATmega128 单片机的一 些寄存器的配置和访问功能函 数	
		fun.c		
		timer.h	实现对 ATmega128 单片机中 0 号计数器的控制，并且虚拟出 两个计数器	
		timer.c		

	射频驱动	cc1000.h	提供射频芯片的底层驱动和配置，以及 ATmega128 单片机与 CC1000 之间的通讯	MAC 库
		cc1000.c		
PHY 代码	物理层	radiocontrol.h	调用 CC1000 实现数据传输的功能模块，可以实现 RF 的睡眠	
		radiocontrol.c		
		physical.h	负责将数据帧以字节为单位提供给 radiocontrol，并且负责将收到的数据打包发送给 MAC 层	
		physical.c		
MAC 代码	MAC 层	macClock.h	1 号计数器的控制模块，主要用于 MAC 层的超时计数	
		macClock.c		
		mac.h	MAC 层模块，负责 mac 层的功能和调度，包括 RTS—CTS 机制，ACK 机制，透明的传输数据包的功能	
		mac.c		

(1) MAC 数据包格式说明

下面是 message.h 中的部分代码，相关的数据包的格式在该文件中定义。

```
#define MAX_PKT_LEN 100
#define MIN_PKT_LEN 7
#define MAC_HEADER_LEN 7
#define OS_DEFAULT_GROUP 1
#define MAC_CTRL_LEN 7
typedef struct{
    uint8_t length;                // 包总长度
    uint8_t type;                  // 包类型
    uint8_t toAddr;                // 目的地址
    uint8_t fromAddr;              // 源地址
    uint8_t group;                 // 群号
    int8_t data[MAX_PKT_LEN - MAC_HEADER_LEN]; // MAC 包数据部分
    int16_t crc;                   // CRC16
} OSMACMsg;
typedef OSMACMsg* OSMACMsgPtr;
// MAC 数据包类型
```

length,type,toAddr,fromAddr,group 这 5 个字节是数据包的 MAC 头，length 是个比较重要的部分，它表示包的长度，由#define MAC_HEADER_LEN 7 这段代码可以看出在设计的时候 crc 的 2 个字节是计算在 MAC 头中的，对于 length 的计算需要先说明一下 data 数组。

data[MAX_PKT_LEN - MAC_HEADER_LEN]才是用户可以自由发挥的地方，在这里用户可以定义自己的数据包格式，GAINS433MAC 库对数据包是透明的，数据包的解析和填充过程是在外部完成的，不过数据包要放在 data 数组中。比如我定义了一个数据包格式（这是我们下一实验“星形网络实验”的数据包格式）：

```
#define BUFFER_LEN    10

typedef struct{
    uint8_t seqNo;
    int16_t data[BUFFER_LEN];
}SensorMsg;
```

下面是填充 data 数组的代码：

```
SensorMsg *pack;
pack = (SensorMsg *)LLCmsg[LLCcurrentMsg].data; //LLCmsg 是 OSMACMsg 类型
pack->seqNo = LLCpacketNumber;
pack->data[LLCpacketReadingNumber++] = data;
```

对于这个数据包来说，length 的计算应该是 sizeof(SensorMsg) + MAC_HEADER_LEN，这里有一点要强调的就是 length 不是一次就算出来的，只有在 MAC 层中才会加入 MAC_HEADER_LEN，这会在后面介绍 GAINS433MAC 库的时候介绍。

type 字节表示数据包的类型，这都是在 GAINS433MAC 库中定义的，分为下面几种，在库中用枚举类型进行了定义：enum { RTS_PKT, CTS_PKT, DATA_PKT, ACK_PKT }。toAddr 和 fromAddr 字段表示源地址和目的地址，group 字段表示组号，MAC 中设定为 0x01，一般来说 group 字段是用不到的，可以不管，crc 字段是用来做检校用的。

(2) 全局变量的说明以及 GAINS433MAC 库的一些配置参数说明

一些重要的全局变量放在 global.h 文件中，该文件主要是放置一些和 MAC 的配置有关的全局变量，下面是该文件中一些变量的意义：

变量名	用途	示例
POWERLEVEL	用来设定 CC1000 射频芯片的输出功率，附录表 a-1 是 CC1000 的手册提供的数据，需要按照要求设定，具体的可以参考 CC1000 的用户手册，需要注意的是要使用 16 进制。	POWERLEVEL = 0X01; //输出功耗设定为-20dBm
SINKNODE	设置 Sink(即汇集)节点的 ID 号	#define SINKNODE 1 //Sink 节点为 1 号节点
OS_LOCAL_ADDRESS	设置本节点的 ID 号；如果该 ID 号和 SINKNODE 相同，该节点被设置为 Sink 节点	//下面的行将本节点的 ID 号设置为 6 OS_LOCAL_ADDRESS = 6
OS_BCAST_ADDR	设置广播地址，用于广播时使用	//将广播地址设置为 0xEE OS_BCAST_ADDR = 0xEE;

ACK_ON ACKTIME	这两个是一对，配套使用，ACK_ON 表示是否使用 ACK 机制，如果设置为 1 表示使用 ACK 机制，为 0 表示不使用 ACK 机制。如果选择使用 ACK 机制，那么 ACKTIME 的参数就有效了，ACKTIME 代表的时间槽的长度为 48ms，提供这个时间槽的是 1 号计数器的 A 匹配中断，时间槽的长度可以修改。	//等待时间 = 240ms ACK_ON = 0; ACKTIME = 5;
-------------------	---	---

(3) GAINS433MAC 库接口函数介绍

GAINS433MAC 库给用户提供了丰富的接口，便于用户自己开发。GAINSMAC433 库包含了三个模块，分别是 mac，physical 和 radiocontrol，对外接口主要由 mac 模块提供。radiocontrol 模块在初始化的时候会调用 cc1000 模块完成射频芯片的一些配置工作，在 TX 和 RX 切换的时候也会调用 cc1000 模块（为了方便，cc1000 模块也被封装在库中），还有一点需要说明的是 radiocontrol 模块在进行载波监听的时候需要使用 ADC 转换部件。ADC 转换结束的时候会回调 radiocontrol 模块中的 adiocontrolRSSIADCDataReady 函数，在该函数中会根据 ADC 转换获得的数据来判断信道是否空闲，实现这个功能需要占用硬件上的 ADC0 通道。下面对相关的接口函数进行简单的介绍，介绍分为两部分：

1) 入口函数

入口函数也就是可以供用户调用的功能接口，主要包含两个函数，分别是支持单播的函数 MACUnicastMsg 和支持广播的函数 MACBroadcastMsg，另外还有 MAC 层的初始化函数 MACInit。这里解释一下单播的概念，所谓单播就是将数据包发送给指定的节点，由于无线信道的特点，在物理世界电磁波肯定采用的是广播的方式，在广播域内的其它的节点也可以收到该数据包，但是只有指定的节点才接收该数据包，其它的节点直接丢弃该数据包。下面是这两个函数的详细说明：

● MACBroadcastMsg(void* data, uint8_t length)

```

/*****
*功能描述：发送广播数据包的接口函数
*参数说明：包地址，包长度(MAC 包数据部分的长度)
*返回值：
*****/
result_t MACBroadcastMsg(void* data, uint8_t length)
{
    char intEnabled;

    // 错误的发送方式
    if (data == 0 || length == 0 || length > MAX_PKT_LEN - MAC_HEADER_LEN) {
        return FAIL;
    }
}

```

```

    }
    if(state != IDLE ) return FAIL;
    // 设置发送请求标识
    intEnabled = inp(SREG) & 0x80;
    cli();

    if ((txRequest == 0)&&(AckWaitTime == 0)) {
        txRequest = 1;
        if (intEnabled) sei();
    } else {
        if (intEnabled) sei();
        return FAIL;
    }

    dataPkt = (OSMACMsgPtr)data;
    txPktLen = length + MAC_HEADER_LEN;
    // 数据包包头设置
    dataPkt->type = DATA_PKT;
    dataPkt->toAddr = OS_BCAST_ADDR;
    dataPkt->fromAddr = OS_LOCAL_ADDRESS;
    dataPkt->group = 0x01;

    tryToSend();

    return SUCCESS;
}

```

- MACUnicastMsg(void* data, uint8_t length, uint8_t toAddr,char csma_caflag)

关于 csma_caflag 参数需要说明一下，csma_caflag 为 1 表示该数据包的传输采用 CSMA/CA 退避机制，csma_caflag 为 0 表示该数据包的传输不使用 CSMA/CA 退避机制，使用者可以根据需要自己选择。

```

/*****
*功能描述：发送单播数据包的接口函数
*参数说明：包地址，包长度(包括包头)，目的地址，CSMA/CA 标示
*返回值：
*****/
result_t MACUnicastMsg(void* data, uint8_t length, uint8_t toAddr,char csma_caflag)
{
    char intEnabled;

```

```

// 错误的发送方式
if (data == 0 || length == 0 || length > MAX_PKT_LEN - MAC_HEADER_LEN) {
    return FAIL;
}
if(state != IDLE ) return FAIL;
// 设置发送请求标识
intEnabled = inp(SREG) & 0x80;
cli();

if ((txRequest == 0)&&(AckWaitTime == 0)) { //MAC 占用标志和时间标志，时间标志用于 ACK 等待时对 MAC 的占用，MAC 采用的是阻塞的模式
    txRequest = 1;
    if (intEnabled) sei();
} else {
    if (intEnabled) sei();
    return FAIL;
}

dataPkt = (OSMACMsgPtr)data;
txPktLen = length + MAC_HEADER_LEN;
// 数据包包头设置
dataPkt->type = DATA_PKT;
dataPkt->toAddr = toAddr;
dataPkt->fromAddr = OS_LOCAL_ADDRESS;
dataPkt->group = 0x01;

if(csma_caflag){
    tryToSend(); //进行载波监听
}else{
    startBcast();//不进行载波监听，直接发送
}

return SUCCESS;
}

```

这里需要说明一下 $txPktLen = length + MAC_HEADER_LEN$ ， $txPktLen$ 表示的是 mac 数据包的长度，而传给两个入口函数的 $length$ 参数仅仅是 $data[MAX_PKT_LEN - MAC_HEADER_LEN]$ 数组中有效数据的长度，也就是说传给接口函数的数据包长度的大小是不应该包含 MAC 字段部分的长度的，可以参考 5.1.3 节关于 MAC 数据包格式的说明。

这里还需要说明一下 `MACClockFire` 函数，该函数被 MAC 层的时钟中断函数调用，主要是用于超时计数，现在我们设定的中断时间间隔是 32ms，用户可以修改 `macClock.c` 中的函数 `ClockStart` 里面的 `SCALE_1ms` 和 `INTERVAL_1ms` 两个参数来修改中断时间间隔。

2) 回调函数

回调函数是 `GAINSMAC433` 库对外的信息接口，通过回调函数外部模块可以被动地获得 MAC 层的一些运行状态，需要注意的是回调函数需要用户来实现，回调函数中最主要的几个函数是 `ReceiveDone`，`TransmitDone`，`SendFail`，`GetFreeQueueLength`，由于这几个函数需要用户来实现，我们只是简单介绍一下其功能。

- `result_t ReceiveDone(OSMACMsgPtr packet);`

该函数的参数是被接收到的数据包指针，该函数是 MAC 层正确接收到一个完整的数据包后向上层提交数据包的一个响应，能通过 MAC 层的数据包要么是广播数据包，要么是指定发往本节点的数据包，其它非发往本节点的单播数据包会被 MAC 层过滤掉，上层看不见这些数据包。

- `result_t TransmitDone(OSMACMsgPtr msg);`

该函数的参数是被传送的数据包指针，该函数是 MAC 层完成一个数据包的传输后通知上层的函数，对于广播和单播它们的意义有所不同：在广播方式下，MAC 层只要将数据包发送出去了，就表示数据发送完成了，可以调用 `TransmitDone` 函数来通知上层数据发送完成；在单播方式下，如果采取不需要 ACK 的方式，那么其工作的机理和广播一样，但在有 ACK 方式下，MAC 层只有在收到 ACK 信息以后，才会调用 `TransmitDone` 函数来通知上层数据发送完成。

- `void SendFail(OSMACMsgPtr receivemsg);`

该函数的参数是发送失败的数据包指针，用于通知上层发送失败，它只有在采用单播并且同时使用 ACK 机制的时候才有效，在 ACK 等待超时的情况下 MAC 层会回调该函数，通知上层数据包发送失败。

对于 `SendFail` 这个函数读者可能会有异议，因为在 MAC 层引起数据包发送失败的因素很多，设计中调用该函数的时机只是针对最特殊的问题，在 MAC 层被占用的情况下同样会引起数据包的传送失败。需要讨论一下数据包在 MAC 层被丢掉的问题，这里只是考虑了比较特殊的情况而没有考虑其它情况，比如 MAC 层被占用等，因为这个特殊的情况比较好掌握，可以通过状态机来描述，但是其它的一些情况很难描述，而且比较复杂。那么对于其它的情况是如何解决数据包的传输失效的问题呢，这就要提一提 `TransmitDone` 函数了，如果一个数据包没有被传送出去，那就不可能调用 `TransmitDone` 函数，可以给这个数据包加入一个标记，在数据包可用于发送的时候给其标示相关处理，在 `TransmitDone` 回调函数中清除标示位。如果发送失败，那么不会回调 `TransmitDone` 函数，那么标示位还在，在下一个发送任务来到的时候可以重新发送该数据包。这里谈到的发送任务的概念和操作系统的任务时间片的概念类似，比如 48ms 启动一次发送任务，用户可以在任何发送任务来到的时候，调用 MAC 提供的广播函数 `MACBroadcastMsg` 和单播函数 `MACUnicastMsg`，但是一次调用并不一定能够将数据包发送出去，需要周期性的执行发送任务来弥补发送失败的可能。

由上面的讨论，对于一般的应用场合 `SendFail` 函数没有存在的必要，但是在一些特别的场合，比如 A 点想探测 B 点是否在自己的通讯范围内，那么 A 点可以发一个需要 ACK 的单播数据包给 B，如果 `SendFail` 函数被调用，那么 B 点就不在 A 的通讯范围内。这里还需要说明一下，ACK 等待时间是在数据包被发送出去后才开始计数的，因此回调 `SendFail` 函数，一定是有单播数据包发送出去了。

- `result_t GetFreeQueueLength(void);`

该函数主要是和 ACK 配合使用，在节点收到一个需要转发的数据包的时候，如果节点有空间存储该数据包，那么发送 ACK 消息，如果没有存储空间了，即使收到了该数据包也不发送 ACK 信息。在节点作为路由器，进行数据转发的时候，这个函数变得特别有用，可以提高网络的性能，在不使用该功能的情况下，需要将返回值置为 1。

下面介绍一下控制射频部分休眠与启动的两个函数，这两个函数在 `radiocontrol` 模块中完成，用户可以调用以实现一些特殊的用途。

- 启动射频的函数：`result_t RadiocontrolIdle(void);`

函数原型：

```
/**
 *功能描述：射频控制进入空闲状态，CC1000、SPI 进入接收状态
 *参数说明：无
 *返回值：成功
 */
result_t RadiocontrolIdle()
{
    if (state == IDLE)
        return SUCCESS;
    if (state == SYNC_START) {
        state = IDLE;
        return SUCCESS;
    }
    if (state == SLEEP) { // 打开 CC1000
        CC1000ControlStdControlStart();
    } else {
        * (volatile unsigned char *) (0x0D + 0x20) = 0x00; // 关 SPI 和 SPI 中断
    }
    CC1000ControlRxMode(); // CC1000 设为接收模式
                          // 配置 SPI 为输入
    OSH_MAKE_MISO_INPUT();
    OSH_MAKE_MOSI_INPUT();
    * (volatile unsigned char *) (0x0D + 0x20) = 0xc0; // 开 SPI 和 SPI 中断
    state = IDLE;
```

```

    return SUCCESS;
}

```

- 射频休眠的函数：result_t RadiocontrolSleep(void);

```

/*****
*功能描述：射频控制进入睡眠状态，CC1000、SPI 进入关闭状态
*参数说明：无
*返回值： 成功
*****/
result_t RadiocontrolSleep()
{
    if (state == SLEEP)
        return SUCCESS;

    * (volatile unsigned char *) (0x0D + 0x20) = 0x00;    // 关 SPI 和 SPI 中断
    CC1000ControlStdControlStop();                        // 关 CC1000

    state = SLEEP;
    return SUCCESS;
}

```

六、实验参考程序

(1) 固定路由原理与网络数据包格式的介绍

该网络的实现比较简单，只是在源代码中指定本节点的转发节点号，比如 3-2-sink(sink 节点的 ID 号设为 1)的一个网络数据传输路径，可以作这样的设定：

- 3 号节点每隔 1 秒钟采集一个数据然后将该数据传给 2 号节点；
- 2 号节点接收到 3 号节点的数据包以后转发该数据包到 sink 节点。

首先我们设计网络数据包，一个完整的数据包分为 3 个部分：mac 头部分、网络层部分和应用层数据部分。下面我们介绍一下我们所设计的数据包的详细的数据表现形式：

```

/*****
*整个数据包的格式
*包括 mac 头部分（在这里 mac 头部分包含了 crc 检校位）和数据（data）部分
*****/
typedef struct{
    uint8_t length;                // 包总长度
    uint8_t type;                  // 包类型
    uint8_t toAddr;                // 目的地址
    uint8_t fromAddr;              // 源地址
    uint8_t group;                 // 群号
    int8_t data[MAX_PKT_LEN - MAC_HEADER_LEN]; // MAC 包数据部分
    int16_t crc;                   // CRC16
}

```

```
} OSMACMsg;
```

data 数组部分放入网络层数据和应用层数据，下面是网络层数据的详细的数据表现形式：

```
/******  
* 数据包的网络层部分包含 4 个字段(在 message.h 文件中定义)  
*****/  
typedef struct SHopMsg {  
    //包类型，在我们所讨论的范围内主要是分为了网络控制包类和数据包类两种；  
    uint8_t type;  
    //网络包序号  
    uint16_t seq;  
    //生存时间，该字段用来设定网络包的生存时间  
    uint8_t lifetime;  
    // data 字段主要是用来做一个标记，在填充应用层数据的时候好实现定位  
    //其具体用法在代码分析中会有说明。  
    uint8_t data[1];  
} SHopMsg;
```

下面是应用层的数据包格式，这里需要说明一下的是为了和 snamp 后台的数据包解析相兼容，数据包的相关字段的格式是固定的不能改，不过内容可以改变，应用层的数据包含 5 个部分(在 app.h 文件中定义)。

```
/******  
* 应用层的数据包(在 app.h 文件中定义)  
*****/  
typedef struct SensorMsg {  
    uint8_t type;          //定义传感数据的类型 光还是温度等  
    uint16_t sensorseq;    //应用层的数据包号  
    uint16_t reading;      //表示采集的传感数据的大小，ATmega128 芯片的 adc 采样是 10 位  
    uint8_t src;          //发送数据包的源节点 ID  
    uint8_t passnode[MAXPASS]; //数据包所经过的所有节点的 id 号  
} SensorMsg;
```

(2) 协议的实现与分析

协议的实现主要用到了 GAINS433MAC 库的单播函数 `result_t MACUnicastMsg(void* data, uint8_t length, uint8_t toAddr, char csma_ca)`，其功能是将数据包发送到指定的节点。该部分完整的代码工程在本书配套的光盘中，下面简单的说明一下一些重要函数的功能(下面的函数都在 app.c 主控模块中)：

```

/*****
                                全局函数声明
*****/
result_t Init(void) {
    //下面是配置节点的一些数据，需要在节点初始化之前定义，因为
    //有些数据是在初始化的时候直接赋值给相关的寄存器
    POWERLEVEL = 0xFF; //射频的功率

    OS_LOCAL_ADDRESS = 1; //本地节点的地址
    OS_BCAST_ADDR = 0xEE; //广播地址

    ACK_ON = 0; //使用 ack 机制，1 表示开启，0 表示关闭
    ACKTIME = 4; //ack 等待时间=ACKTIME*48ms
    //选择 4 的理由是和发送成功时的一样的，尽量假设数据包成功，ACK 不成功

    //////////
    LedInit();
    LedRedOff();
    LedGreenOff();
    LedYellowOff();
    //初始化光强传感器
    SensorPhoOStdControlInit();
    //初始化 0 号计数器
    TimerStdControlInit();
    // 初始化 UART debugging
    uartDebug_init();
    MACInit();

    sensorseq = 0;
    networkseq = 0;

    routesensormsgptr = &routesensor;

    return SUCCESS;
}

```

该函数初始化一些 ATmega128 单片机中要使用到的一些硬件资源，包括指示灯，光强传感器，0 号计数器，串口以及无线射频部分。

```

result_t Start(void) {
    SensorPhoOStdControlStart();
}

```

```

    ROUTEGetADCDataStart();
    return SUCCESS;
}

static result_t ROUTEGetADCDataStart(void) {
    TimerTimerStart(1, TIMER_REPEAT, 2000); //定时采集数据
    return SUCCESS;
}

result_t Timer0_0_Fired(void) {
    // 添加时间事件
    return SUCCESS;
}

result_t Timer0_1_Fired(void) {
    //采集光数据
    SensorExternalPhotoADCGetData();
    return SUCCESS;
}

```

上面 4 个函数一起介绍，Start 函数启动光强传感器（主要是给其供电），通过调用 ROUTEGetADCDataStart 函数启动标号为 1 的计数器（该计数器是通过 0 号计数器虚拟出来的，具体介绍在 5.1 节中）。Timer0_0_Fired 和 Timer0_1_Fired 函数是标号为 0 和标号为 1 的两个虚拟计数器的中断调用函数，这里我们使用了标号为 1 的计数器，在相应的 Timer0_1_Fired 中断响应函数中调用了 ADC 采样函数，我们设定的时间是 2000ms 一次采样。

```

/*****
*功能描述：将 ADC 采样得到的数据采用单播的方式发送出去
*参数说明：ADC 采样数据
*返回值：
*****/

static void SendSensorData(uint16_t localsensordata) {
    //发送采集到的数据，暂为空
    SensorMsg *sensordata;
    SHopMsgPtr sHopMsg;
    uint8_t i;
    uint8_t length;

    sHopMsg = (SHopMsgPtr)routesensormsgptr->data;
    sHopMsg->type = SENSOR;
    sHopMsg->seq = networkseq;

```

```

sHopMsg->lifetime = 0; // 本程序中无效的字段统一初始化为 0

sensordata = (SensorMsg *) (sHopMsg->data);
sensordata->type = LIGHT; // 传感数据为光数据
sensordata->sensorseq = sensorseq;
sensordata->reading = localsensordata;
sensordata->src = OS_LOCAL_ADDRESS;

for (i = 0; i < MAXPASS; i++) {
    sensordata->passnode[i] = 0;
}
sensorseq++; // 应用层数据包号自增
sensordata->passnode[0] = OS_LOCAL_ADDRESS;

length = SHOP_HEADER_LEN + sizeof(SensorMsg);

MACUnicastMsg(routesensormsgptr, length, 2, 1);
}

```

上面的函数由 ADC 中断响应函数调用，其功能是将 ADC 采样得到的数据组合成数据包，通过 MACUnicastMsg(routesensormsgptr, length, 2, 1) 函数，将数据包发送给指定的节点（这里指定的是 2 号节点）。这里需要说明一下的是 MACUnicastMsg 函数的参数 length 的计算方法，在作为参数传给 MACUnicastMsg 函数的时候，length 不包含 MAC 头字段的 7 个字节的长度，在 MAC 层中会自动加上 MAC 部分的长度。

```

/*****
*功能描述：非 Sink 节点缓存接收到的数据，并调用发送任务；
            Sink 节点将数据包发往串口；
*参数说明：被发送的数据包的指针
*返回值：
*****/
static void routeReceiveSENSOR(OSMACMsgPtr receivemsg) {
    SHopMsgPtr sHopMsg;
    SensorMsg *sensordata;
    uint8_t i;
    uint8_t j;

    uartMsg = *receivemsg;

    if (OS_LOCAL_ADDRESS != SINKNODE) {
        sHopMsg = (SHopMsgPtr) receivemsg->data;
    }
}

```

```

        sensordata = (SensorMsg *) (sHopMsg->data);
        LifeTime = sHopMsg->lifetime;

        forwardsensor.type = sensordata->type;
        forwardsensor.sensorseq = sensordata->sensorseq;
        forwardsensor.reading = sensordata->reading;
        forwardsensor.src = sensordata->src;

        for (i = 0; i < MAXPASS; i++) {
            forwardsensor.passnode[i] = sensordata->passnode[i];
        }
        j = 0;
        for (i = 0; i < MAXPASS; i++) {
            if (sensordata->passnode[i] != 0) {
                j = j+1;
            } else {
                break;
            }
        }
        if( j < MAXPASS ){ //越界检查
            forwardsensor.passnode[j] = OS_LOCAL_ADDRESS;
        }
        LedGreenToggle();
        OSPostTask(sendreceivesensor);
    } else {
        //如果是 SINK 节点，则写到后台
        LedRedToggle(); //红灯亮，表示基站接收到传感数据包
        uartDebug_txPacket(&uartMsg);
    }
}

```

上面的函数实现对接收到的数据包的分析，通过判断是否是 Sink 节点来决定是缓存转发数据还是通过串口向 pc 提交数据包。

```

/*****
*功能描述：发送接收并缓存的数据包
*参数说明：
*返回值：
*****/

static void sendreceivesensor(void) {
    SHopMsgPtr sHopMsg;

```

```
SensorMsg *sensordata;
uint8_t length;

sHopMsg = (SHopMsgPtr )routesensormsgptr->data;
sHopMsg->type = SENSOR;
sHopMsg->seq = networkseq;
sHopMsg->lifetime = LifeTime;
sensordata = (SensorMsg *) (sHopMsg->data);
*sensordata = forwardsensor;

length = SHOP_HEADER_LEN + sizeof(SensorMsg);
MACUnicastMsg(routesensormsgptr, length, 1,1);
}
```

该函数负责将缓存的数据包发送出去，需要注意的是这个实验的路由是指定的，如果读者想要有所改动的话，需要对 sendreceivesensor 函数中的 MACUnicastMsg 作一些改动。

实验四：WSNs 的 Flooding 实验

一、实验目的

1. 掌握 GAINS433MAC 库的使用
2. 培养利用 GAINS433MAC 库开发高级通讯协议的能力
3. 理解 Flooding 的工作原理
4. 掌握初步的数据处理能力

二、实验设备

- | | |
|--------------------|----|
| 1. PC 机 | 一台 |
| 2. GAINS-3 教学开发平台 | 一套 |
| 3. 软件：WINAVR V4.07 | |
| 4. AVRStudio | |

三、实验内容

使用 GAINS433MAC 库实现一个泛洪协议，具体的方法在实验程序介绍中说明。采集足够的数据（不少于 100 组），并对其进行分析。

四、实验步骤

1. 启动 Programmers Notepad，打开文件“泛洪路由实验”目录下的 router.pnproj 工程。
2. 在 app.c 中的 Init 函数中修改 OS_LOCAL_ADDRESS 的值，分别编译后获得多个 .hex 文件。可以通过修改 POWERLEVEL 的值来修改天线的功率，从而控制传输距离，具体的参数设置见附录中的表 a-1。
3. 将 .hex 下载到节点中。
4. 随机布放节点，并以 ID 最大的节点为数据源，利用 Flooding 在全网广播。**注意：**
 - a) 提供的源代码中指定 3 号节点为数据源节点，请根据实际情况修改；
 - b) 节点数不少于 5 个（包括 Sink 节点）；
 - c) 布放节点的时候，请根据节点的发射功率调整节点间距离，保证网络的连通性。
5. 通过串口线连接 1 号（Sink）节点到 pc，打开所有节点。
6. 启动 SNAMP 可视化后台软件，可以观察到 3 号节点将数据包发送到 sink 节点所经过的路由。
7. 通过串口调试助手，将获取的数据进行分析，要求：
 - a) 分析收到的数据包每个字节的意义；
 - b) 给出每个数据包被转发的次数；
 - c) 给出光照度曲线；

五、实验参考程序

该协议的实现基于泛洪协议的思想，也就是产生数据的节点向全网广播该数据包，收到该数据包的节点也采用广播的方式向全网广播接收到的数据包，原理如图 4-1，如果 Sink 节点接收到该数据包就将其发送给后台 PC。

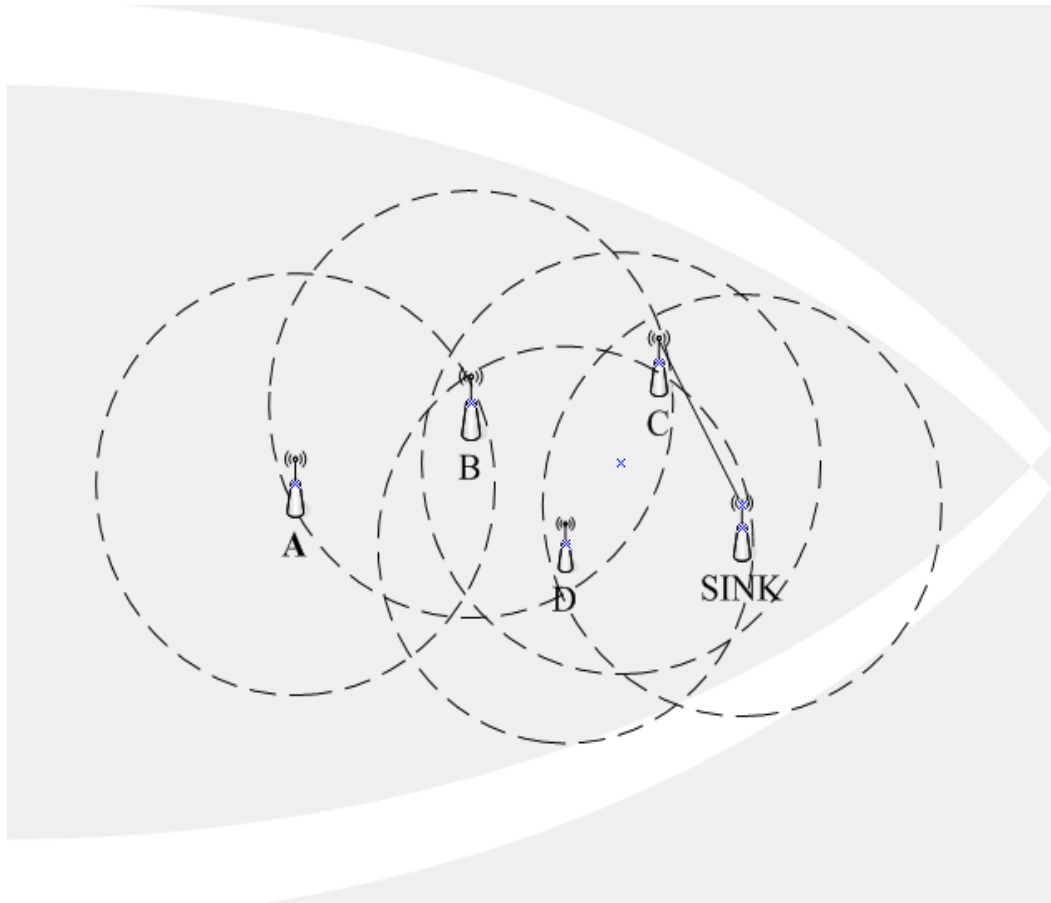


图 4-1 泛洪协议原理图

协议的开发在上面的实验代码基础上面进行扩展，在原有代码的基础上面加入了一些避免循环路由和广播风暴的机制。所谓的广播风暴是由广播引起的，以图 4-1 图示的网络为例，假设 A 为数据源，A 通过广播给 B 传输了一个数据包，那么 B 要转发该数据包，在广播方式下 A, C, D 都会收到 B 点转发的数据包，那么 A, C, D 又要转发该数据包，结果可能是灾难性的。

可以采用数据包登记的机制来避免这种广播效应，每个节点接收到一个数据包后，先要检查自己的登记表，看是否收到过相同的数据包，如果有相同的就不转发，如果没有相同的就转发并且将该数据包登记。数据包的标示由数据包的源节点号以及数据包的包号组成，由于数据包中的 mac 头部分和网络层部分的参数在数据包传输的过程中是可变的，因此只能提取应用层部分的数据，选择 SensorMsg 中的 src 和 sensorseq 作为判断一个数据包的标识。

```
typedef struct SensorMsg {  
    uint8_t type;          //定义传感数据的类型 光还是温度等  
    uint16_t sensorseq;  
    uint16_t reading;  
    uint8_t src;
```

```
uint8_t passnode[MAXPASS];  
} SensorMsg;
```

实验中 3 号节点为数据源，每 2 秒钟 3 号节点产生一个数据包，该数据包通过广播的方式发送出去，其它节点（非 Sink 节点）接收到该数据包后，也是通过广播的方式进行转发。下面是一些主要的函数的介绍和说明(下面的程序都在 app.c 文件中定义):

```
/******  
*功能描述：将 ADC 采样得到的数据采用广播的方式发送出去  
*参数说明：ADC 采样数据  
*返回值：  
*****/  
static void SendSensorData(uint16_t localsensordata) {  
    //发送采集到的数据，暂为空  
    SensorMsg *sensordata;  
    SHopMsgPtr sHopMsg;  
    uint8_t i;  
    uint8_t length;  
  
    sHopMsg = (SHopMsgPtr)routesensormsgptr->data;  
    sHopMsg->type = SENSOR;  
    sHopMsg->seq = networkseq;  
    sHopMsg->lifetime = 0;// 本程序中无效字段统一初始化为 0  
  
    sensordata = (SensorMsg *) (sHopMsg->data);  
    sensordata->type = LIGHT; //传感数据为光数据  
    sensordata->sensorseq = sensorseq;  
    sensordata->reading = localsensordata;  
    sensordata->src = OS_LOCAL_ADDRESS;  
  
    for (i = 0; i < MAXPASS; i++) {  
        sensordata->passnode[i] = 0;  
    }  
    sensorseq++; //应用层数据包号自增  
    sensordata->passnode[0] = OS_LOCAL_ADDRESS;  
  
    length = SHOP_HEADER_LEN + sizeof(SensorMsg);  
    InRecord(sensordata->src, sensordata->sensorseq);  
    MACBroadcastMsg(routesensormsgptr, length);  
}
```

上面的函数的功能和前一个实验的基本一样，不过是将原来的单播方式改为广播方式，同时加入了 InRecord 函数的调用。InRecord 函数的功能是登记一个数据包到本地的数据包登记表，下面是它的代码：

```
/******  
*功能描述：将新的数据包登记到本地的登记表中  
*参数说明：数据包的源节点号，数据包的序号  
*返回值：  
*****/  
  
static void InRecord(uint8_t nodeid,uint16_t sensorseq){  
    record[recordflag].nodeid = nodeid;  
    record[recordflag].sensorseq = sensorseq;  
    recordflag = (recordflag + 1)%MAXPACKET;  
}
```

泛洪协议的实现在指定路由代码基础上面进行扩充，主要是在产生数据包和接收数据包时的一些改变，上面介绍的是对产生数据包的函数的改动，下面介绍对接收数据包的处理函数的改动，下面是相关的代码。

```
/******  
*功能描述：非 Sink 节点缓存接收到的数据，并调用发送任务；  
            Sink 节点将数据包发往串口；  
*参数说明：被发送的数据包的指针  
*返回值：  
*****/  
  
static void routeReceiveSENSOR(OSMACMsgPtr receivemsg) {  
    SHopMsgPtr sHopMsg;  
    SensorMsg *sensordata;  
    uint8_t i;  
    uint8_t j;  
    sHopMsg = (SHopMsgPtr)receivemsg->data;  
    sensordata = (SensorMsg *) (sHopMsg->data);  
    if(1 == CheckInRecord(sensordata->src,sensordata->sensorseq)){  
        //接收到的数据包已经在登记表中了  
        return ;  
    }else{  
        //接收到的数据包不在登记表中，那么在登记表中将该数据包登记  
        InRecord(sensordata->src,sensordata->sensorseq);  
    }  
  
    uartMsg = *receivemsg;  
  
    if (OS_LOCAL_ADDRESS != SINKNODE) {
```

```

        sHopMsg = (SHopMsgPtr )receivemsg->data;
        sensordata = (SensorMsg *) (sHopMsg->data);
        LifeTime = sHopMsg->lifetime;

        forwardsensor.type = sensordata->type;
        forwardsensor.sensorseq = sensordata->sensorseq;
        forwardsensor.reading = sensordata->reading;
        forwardsensor.src = sensordata->src;

        for (i = 0; i < MAXPASS; i++) {
            forwardsensor.passnode[i] = sensordata->passnode[i];
        }
        j = 0;
        for (i = 0; i < MAXPASS; i++) {
            if (sensordata->passnode[i] != 0) {
                j = j+1;
            } else {
                break;
            }
        }
        if( j < MAXPASS ){ //越界检查
            forwardsensor.passnode[j] = OS_LOCAL_ADDRESS;
        }
        LedGreenToggle();
        OSPostTask(sendreceivesensor);
    } else {
        //如果是 SINK 节点，则写到后台
        LedRedToggle(); //红灯亮，表示基站接收到传感数据包
        uartDebug_txPacket(&uartMsg);
    }
}

/*****
*功能描述：发送接收并缓存的数据包
*参数说明：
*返回值：
*****/

static void sendreceivesensor(void) {
    SHopMsgPtr sHopMsg;
    SensorMsg *sensordata;

```

```
uint8_t length;

sHopMsg = (SHopMsgPtr )routesensormsgptr->data;
sHopMsg->type = SENSOR;
sHopMsg->seq = networkseq;
sHopMsg->lifetime = LifeTime;
sensordata = (SensorMsg *) (sHopMsg->data);
*sensordata = forwardsensor;

length = SHOP_HEADER_LEN + sizeof(SensorMsg);
MACBroadcastMsg(routesensormsgptr, length);
}
```

这两个接收数据包后的处理函数，首先对接收到的数据包进行检查，如果转发过了就不处理，否则就在本节点登记该数据包，然后转发，对应的转发方式也是采用广播的方法。

附录

表 a-1 输出功率表

输出功率(dBm)	发射频率 433Mhz		发射频率 868Mhz	
	PA_POW 寄存器 配置参数(hex)	电 流 消 耗 量 (mA)	PA_POW 寄存器 配置参数(hex)	电 流 消 耗 量 (mA)
-20	01	5.3	02	8.6
-19	01	6.9	02	8.8
-18	02	7.1	03	9.0
-17	02	7.1	03	9.0
-16	02	7.1	04	9.1
-15	03	7.4	05	9.3
-14	03	7.4	05	9.3
-13	03	7.4	06	9.5
-12	04	7.6	07	9.7
-11	04	7.6	08	9.9
-10	05	7.9	09	10.1
-9	05	7.9	0B	10.4
-8	06	8.2	0C	10.6
-7	07	8.4	0D	10.8
-6	08	8.7	0F	11.1
-5	09	8.9	40	13.8
-4	0A	9.6	50	14.5
-3	0B	9.4	50	14.5
-2	0C	9.7	60	15.1
-1	0E	10.2	70	15.8
0	0F	10.4	80	16.8
1	40	11.8	90	17.2
2	50	12.8	B0	18.5
3	50	12.8	C0	19.2
4	60	13.8	F0	21.3
5	70	14.8	FF	25.4
6	80	15.8		
7	90	16.8		
8	C0	20.0		
9	E0	22.1		
10	FF	26.7		